



Trabajo Práctico

MANEJADOR DE RECURSOS DISTRIBUIDOS PARA HPC

Duración Estimada Individual: 25-30 horas.

Resumen

El objetivo de este proyecto es diseñar e implementar un **middleware distribuido** que gestione recursos (CPUs, memoria, GPUs) de un clúster HPC simulado. Cada equipo construirá un nodo autónomo que coopera con los nodos de otros equipos. Para abordar la complejidad, el equipo se dividirá en cuatro roles especializados (comunicaciones C, gestión de recursos, lógica Erlang, integración y pruebas). Se exigirá el uso de **epoll** (en C) en el servidor de recursos, mientras que la lógica de planificación y coordinación se implementará en **Erlang**. Al final, todos los nodos deberán interoperar mediante un protocolo común.

§1. Objetivos pedagógicos

- Comprender los desafíos de la sincronización distribuida y el riesgo de interbloqueo.
- Utilizar **epoll** para gestionar cientos de conexiones TCP de forma no bloqueante en C.
- Desarrollar un sistema concurrente en Erlang, comunicándose con un servidor en C. Ensamblar módulos escritos en C (eventos) y Erlang (actores) mediante un protocolo claramente definido.
- Implementar un protocolo de asignación de recursos entre nodos heterogéneos.
- Aplicar un algoritmo de prevención o detección de deadlock distribuido.
- Lograr interoperabilidad entre diferentes implementaciones (distintos equipos).

§2. Objetivos del trabajo en equipo y demostración en el laboratorio

Además de los objetivos técnicos, el proyecto está diseñado para fomentar las siguientes competencias colaborativas:

- Coordinación y planificación:** Adquirir experiencia para el trabajo colaborativo e intercambio de ideas. Gestionar dependencias cruzadas entre las diferentes tareas realizadas por cada miembro del equipo.
- Depuración conjunta:** aislar fallos en sistemas distribuidos usando logs, trazas de red y tests automatizados.
- Pruebas conjuntas:** validar el nodo propio con nodos de otros equipos, diseñando un plan de pruebas colaborativo.
- Responsabilidad individual:** cada miembro del equipo tiene entregables específicos que afectan al éxito del proyecto.
- Documentación para terceros:** redactar un **README.md** y diagramas que permitan a cualquier otro equipo compilar y ejecutar el nodo sin ayuda adicional.

- f) **Defensa colectiva:** exponer y justificar el funcionamiento global del sistema, respondiendo preguntas sobre cualquier aspecto del mismo.

§3. Descripción general

Se debe implementar un sistema que contará con dos componentes, a saber:

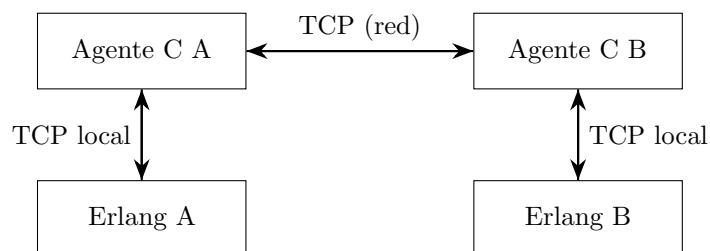
a) **Resource Manager Agent (en C).**

- Cada agente C actúa simultáneamente como servidor y como cliente:
 - Como servidor escucha en un puerto TCP público para recibir peticiones de otros agentes remotos.
 - Como cliente cuando su planificador Erlang necesita un recurso remoto, el agente C inicia una conexión TCP saliente hacia el agente remoto.
- Administra recursos locales (ej. `cpu:4, mem:8192 MB, gpu:1`).
- Mantiene conexiones salientes hacia otros agentes.
- Utiliza `epoll` para manejo concurrente no bloqueante.
- Ofrece una interfaz local (mismo puerto, pero sólo acepta conexiones desde `localhost`) para que el proceso Erlang le envíe comandos.
- Implementa un mecanismo de **descubrimiento dinámico de nodos** mediante UDP broadcast.

b) **Job Scheduler (en Erlang):**

- Se conecta al servidor C localmente (TCP a `localhost:PUERTO_C`).
- Toma decisiones sobre qué *job* lanzar y qué recursos solicitar en cada nodo.
- Implementa explícitamente una estrategia contra deadlocks.
- Simula carga de trabajo: envía peticiones de recursos y libera después de un tiempo.

§3.1. Arquitectura general



Además:

[C Agent] ↔ (UDP broadcast/multicast) ↔ [descubrimiento de nodos]

§4. Protocolos de comunicación

§4.1. Protocolo entre Nodos C (línea ASCII, terminada en `\n`)

Mensaje entre Agentes:

```
RESERVE <job_id> <resource_name> <amount>
GRANTED <job_id>
DENIED <job_id>
RELEASE <job_id> <resource_name> <amount>
```

Donde

- `job_id`: entero único generado por el planificador Erlang.
- `resource_name`: cadena como `cpu`, `mem`, `gpu`.
- `amount`: cantidad solicitada (entero).

§4.2. Interfaz local (Erlang ↔ C)

El agente C acepta comandos desde conexiones TCP originadas en `localhost` en el mismo puerto. Formato (Comandos posibles desde Erlang):

```
JOB_REQUEST <job_id> [ @host:res:amount ... ]
JOB_RELEASE <job_id>
JOB_STATUS <job_id>
```

Respuestas posibles del agente C hacia Erlang:

```
JOB_GRANTED <job_id>
JOB_DENIED <job_id>
JOB_TIMEOUT <job_id>
```

Ejemplo:

```
# Nodo A recibe JOB_REQUEST de su Erlang:
    JOB_REQUEST 1001 @192.168.1.2:cpu:2 @192.168.1.3:gpu:1
# A envia a 192.168.1.2:
    RESERVE 1001 cpu 2
# 192.168.1.2 responde:
    GRANTED 1001
# A envia a 192.168.1.3:
    RESERVE 1001 gpu 1
# 192.168.1.3 responde:
    GRANTED 1001
# A responde a Erlang:
    JOB_GRANTED 1001
```

§5. Requerimientos funcionales detallados

§5.1. Servidor en C con epoll

- El servidor debe abrir dos sockets de escucha en el mismo puerto:
 - a) Uno para conexiones entrantes de otros agentes (dirección pública).
 - b) Otro para la conexión local desde Erlang (solo `127.0.0.1`).
- Agregar todos los descriptores de sockets (escucha, conexiones establecidas, temporizadores opcionales). Se procesarán eventos `EPOLLIN` y `EPOLLOUT` sin bloqueos. Para escritura
- Opcional: usar `timerfd` para implementar timeouts en reservas pendientes.

- Gestión de recursos locales:
 - Cada recurso tiene una capacidad total, una cantidad disponible y una cola FIFO de solicitudes pendientes (identificadas por `job_id` y socket de origen).
 - Al recibir **RESERVE** sobre un recurso local:
 - Si hay suficiente disponible, se descuenta y se responde **GRANTED**.
 - Si no, se encola la solicitud.
 - Al recibir **RELEASE** se libera la cantidad, se descuenta del job y se atienden solicitudes encoladas en orden.
- El agente debe mantener una tabla de jobs activos con los recursos concedidos (para poder liberarlos correctamente ante una desconexión inesperada).

§5.2. Lógica del planificador en Erlang

El planificador deberá:

- Conectarse localmente al agente C.
- Consultar al agente C local sobre la lista de nodos participantes (IP, puerto, recursos disponibles). El planificador Erlang consulta al agente C local con el comando: `GET_NODES`.
El agente C responde con una lista de nodos vivos en formato texto, por ejemplo:
`NODES 192.168.1.10:8100:cpu:4:mem:8192:gpu:1;192.168.1.11:8101:cpu:2:mem:40961`
- Lanzar simultáneamente varios jobs que soliciten recursos en múltiples nodos.
- Implementar alguna estrategia para evitar o prevenir deadlocks distribuidos.
- Registrar en un archivo de log las concesiones, denegaciones y eventos de deadlock.

§5.3. Interoperabilidad entre los agentes

El sistema debe ser capaz de descubrir y mantener una lista actualizada de los nodos activos en la red. Esto simula entornos reales donde los recursos pueden aparecer o desaparecer dinámicamente.

Cada agente C debe implementar las siguientes funcionalidades:

- Anuncio periódico de presencia usando UDP (broadcast).
El mensaje tiene el formato: `ANNOUNCE <IP> <puerto> <recursos>`
Ejemplo: `ANNOUNCE 192.168.1.10 8100 cpu:4 mem:8192 gpu:1`
- Mantenimiento de la tabla de nodos: Cada agente mantiene en memoria una tabla con las entradas: IP, puerto, recursos, timestamp del último anuncio recibido. Si no se recibe un anuncio de un nodo durante 15 segundos, se considera caído y se elimina de la tabla. La tabla se utiliza para decidir a qué nodos enviar solicitudes **RESERVE**
- Arranque inicial: Al iniciar, el nodo envía inmediatamente un anuncio y espera durante 2 segundos para recibir anuncios de otros nodos ya activos. El socket UDP se agrega al conjunto de epoll para eventos **EPOLLIN**. Luego comienza a atender peticiones normales.

¹El formato queda a criterio del equipo de desarrollo ya que sólo lo usa el cliente de Erlang. El planificador usa esa lista para construir las solicitudes `JOB_REQUEST` (puede elegir cualquier nodo que tenga los recursos necesarios).

§6. Escenario de deadlock distribuido

Con dos nodos:

- Nodo A: 2 CPUs, 8 GB RAM, 0 GPU.
- Nodo B: 2 CPUs, 4 GB RAM, 1 GPU.

Jobs:

- Job1 (desde A): necesita 2 CPUs de A y 1 GPU de B.
- Job2 (desde B): necesita 1 GPU de B y 2 CPUs de A.

Si ambos planificadores envían solicitudes casi al mismo tiempo:

- a) Agente A concede 2 CPUs a Job1.
- b) Agente B concede 1 GPU a Job2.
- c) Luego cada uno espera el recurso del otro → **deadlock**.

Deberán demostrar que su implementación evita o resuelve esta situación.

§7. Roles sugeridas dentro del equipo (4 personas)

El trabajo podría dividirse en los siguientes roles. Cada miembro es responsable principal de su módulo, pero debe colaborar en la integración y comprender el sistema completo.

Rol	Responsabilidades principales	Tecnologías
Ingeniero de comunicaciones en C	Servidor C con <code>epoll</code> : conexiones entrantes/salientes, lectura/escritura no bloqueante, buffers, detección de cierres.	C, <code>epoll</code> , sockets TCP, UDP
Gestor de recursos y estado	Lógica de recursos locales (estructuras, colas FIFO, asignación y liberación), tabla de jobs activos, respuestas <code>GRANTED/DENIED</code> .	C, estructuras de datos, concurrencia
Planificador y lógica anti-deadlock	Proceso Erlang: conexión al agente C, generación de jobs, estrategia de prevención/detección de deadlock.	Erlang, concurrencia
Integración, pruebas e interoperabilidad	Comunicación Erlang-C, scripts de prueba, documentación (README, diagramas). Coordinación con otros equipos.	Bash, integración, documentación

§8. Recomendaciones

- Usar Git con ramas por rol y pull requests.
- Realizar reuniones breves diarias (15 min) para sincronizar.
- Usar un planificador de tareas tipo Trello.
- Probar temprano la comunicación entre Erlang y el agente C usando `nc -l` como simulador.
- Para enviar mensajes desde Erlang al agente C, utilice `gen_tcp:connect/3` y `gen_tcp:send/2`.
- El agente C debe manejar la desconexión inesperada (ej. SIGPIPE) de un nodo remoto (liberar recursos asociados a los jobs de ese nodo).
- Los `job_id` pueden generarse con `erlang:unique_integer()` en el planificador.

- Es muy recomendable implementar **timeouts** en el agente C para eliminar reservas que nunca se completan (por ejemplo, mediante **timerfd**).

§9. Entregables

- a) Código fuente completo de ambos componentes (C y Erlang) con comentarios en inglés o español.
- b) **Makefile** que compile el servidor C.
- c) Archivo **README.md** con instrucciones de compilación y ejecución.
- d) Script de prueba **test_deadlock.sh** que ejecute dos instancias locales (en puertos diferentes) y verifique el comportamiento.
- e) Informe de equipo (4-6 páginas) que explique el diseño, problemas encontrados y soluciones. Además:
 - Roles de cada miembro y contribuciones.
 - Estrategia elegida contra deadlock, justificación y ejemplo paso a paso que demuestre cómo se evita un deadlock con dos nodos.
 - Diagrama de secuencia de la comunicación (local y remota).

§10. Modo de entrega

- Se debe entregar un sistema robusto, eficiente y bien documentado.
- Entrega: github y campus virtual (código + README + script + informe) y demostración presencial en laboratorio.
- La demostración será obligatoria.
- En la demostración final, cada miembro deberá explicar su parte y responder preguntas sobre el sistema completo.